

Training Neural Networks

KAUST–Mawhiba IOAI Training

Week 2 · Day 7



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

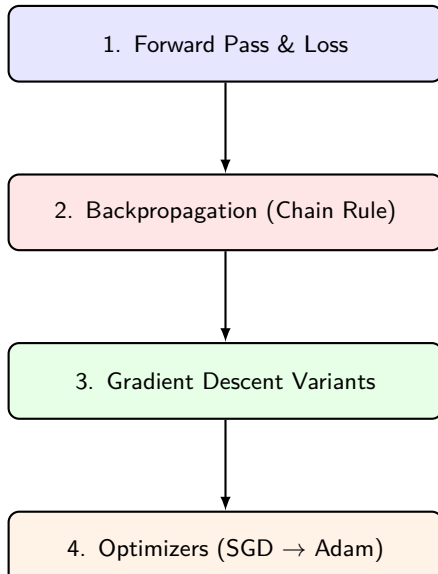
April 29, 2026

1. Recap: From Architecture to Training
2. Forward Pass & Loss
3. Backpropagation (Chain Rule)
4. Gradient Descent: Batch, Mini-Batch, Stochastic
5. Training Steps, Epochs & Shuffling
6. Optimizers: Momentum, AdaGrad, RMSProp, Adam, AdamW
7. Mini-Batch Training in PyTorch (DataLoaders)
8. Training on CPU vs GPU

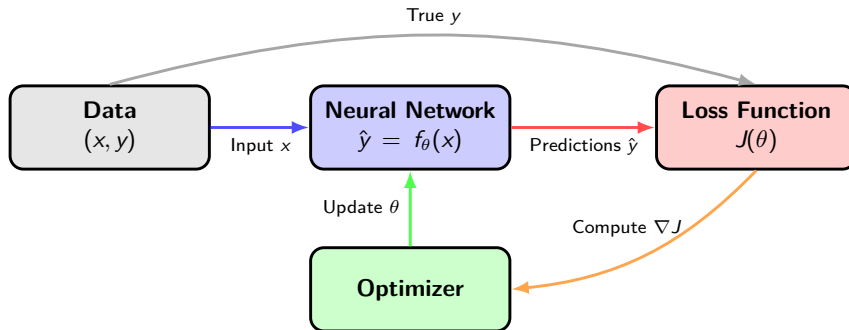
Yesterday you learned the *anatomy* of neural networks:

- ▶ **Perceptron** – a single neuron: $\hat{y} = \sigma(w^T x + b)$
- ▶ **Activation functions** – ReLU, Sigmoid, Tanh
- ▶ **MLP** – stacking layers: Input $\rightarrow$ Hidden $\rightarrow$ Output
- ▶ **PyTorch** – tensors, `nn.Module`, forward pass

Today's question: We can build a network and run a forward pass.
But *how does the network actually learn?*



Neural Networks as Part of a Learning System



To train this model we repeat two phases:

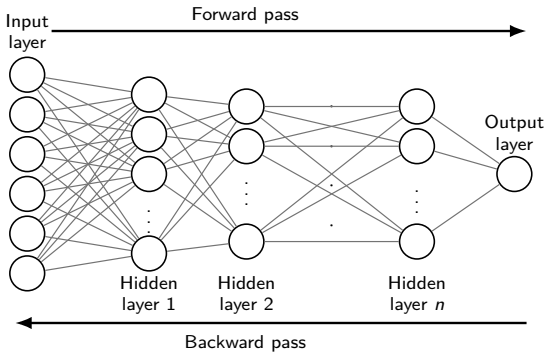
Forward Pass:

- ▶ Data $\rightarrow$ Model $\rightarrow$ Predictions
- ▶ Compute Loss

Backward Pass:

- ▶ Calculate gradients
- ▶ Update parameters

Let's discuss them in detail.



1. Forward Pass

Goal: Compute the network output $\hat{y}$ from input x , then measure the error.

What happens (layer by layer):

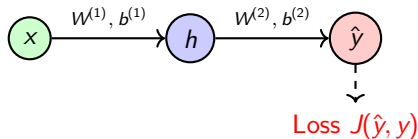
1. Start with input: $a^{(0)} = x$.
2. For each layer ℓ :

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$$

$$a^{(\ell)} = \sigma(z^{(\ell)})$$

3. Final layer gives the prediction:
 $\hat{y} = a^{(L)}$.
4. **Loss:** $J(\theta)$ compares $\hat{y}$ with the true target y .

Forward pass = just prediction (no learning yet).

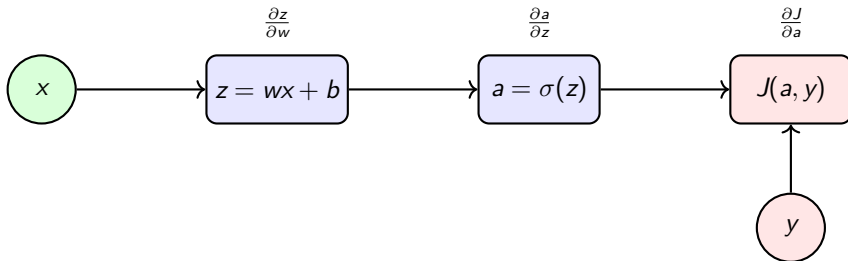


2. Backward Pass (Backpropagation)

Goal: Given feedback from the loss, calculate gradients for all weights, then update them via Gradient Descent.

By applying the **Chain Rule**, we multiply these partial derivatives to derive the gradient for every parameter.

We move from the end (the loss) **backwards** towards the beginning, passing through each weight to get its gradient. This is called **Backpropagation**.



Chain Rule Example: Compute $\frac{\partial J}{\partial w}$

Assume one neuron:

$$z = wx + b, \quad a = \sigma(z), \quad J = \frac{1}{2}(a - y)^2$$

Chain Rule (backprop)

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Now compute each piece:

$$\frac{\partial J}{\partial a} = (a - y) \quad \frac{\partial a}{\partial z} = \sigma(z)(1 - \sigma(z)) = a(1 - a) \quad \frac{\partial z}{\partial w} = x$$

Final result

$$\frac{\partial J}{\partial w} = (a - y) a(1 - a) x$$

What do we do with $\frac{\partial J}{\partial w}$? We use Gradient Descent to update the weights.

Parameter Update Rule (via Gradient Descent)

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w} \quad b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

- ▶ α : learning rate (step size).
- ▶ We compute $\frac{\partial J}{\partial(\cdot)}$ for **every** parameter using backprop.

What do we do with $\frac{\partial J}{\partial w}$? We use Gradient Descent to update the weights.

Parameter Update Rule (via Gradient Descent)

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w} \quad b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

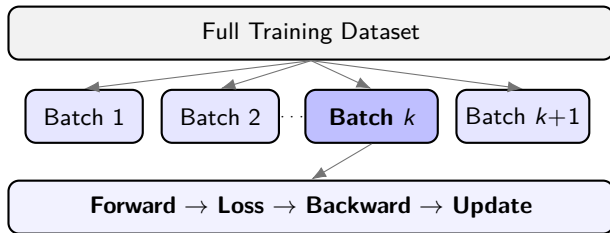
- ▶ α : learning rate (step size).
- ▶ We compute $\frac{\partial J}{\partial(\cdot)}$ for **every** parameter using backprop.

Q: Do we do this update using **all data at once**?

Training Step = One Update

We do not train using the full dataset at once.

Instead, we take a **small batch** (mini-dataset) and do one update:

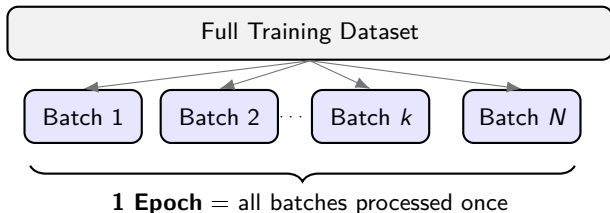


- ▶ This single update is called a **training step** (or **iteration**).
- ▶ **Batch size** = how many samples are used in one step.

*Next: repeat this step for **all** batches $\Rightarrow$ one full **epoch**.*

Epoch = One Full Pass Over the Dataset

Dataset → split into **batches** → each batch makes **one step**.



- Steps per epoch $\approx \left\lceil \frac{\text{\#samples}}{\text{batch size}} \right\rceil$.
- Training usually runs for **many epochs**.

*Next: before the next epoch, we usually **shuffle** the dataset.*

Why Shuffle Between Epochs?

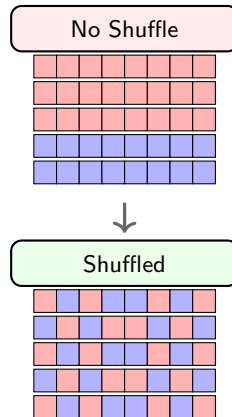
At the start of each epoch, we shuffle the dataset to randomize batch composition.

Without shuffling:

- ▶ Batches see same patterns repeatedly
- ▶ Similar samples grouped together
- ▶ Biased gradient estimates

With shuffling:

- ▶ Each batch is a better mix
- ▶ More stable learning
- ▶ Better generalization



1. The Model (f_{θ})

Design Choice:
Neural Networks
(Architecture)

2. Loss Function (J)

Design Choice:
MSE / Cross-Entropy

3. Optimizer

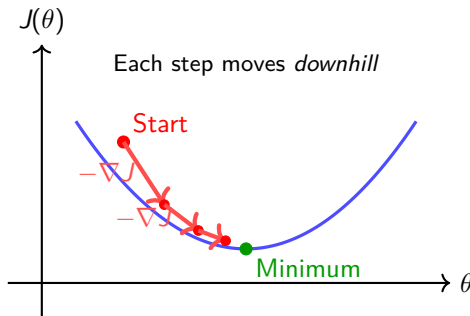
Design Choice:
SGD, Adam, RMSProp

Depends on:
Stability & Con-
vergence Speed

We defined the Models (1) and the Losses (2).
Now, Let's discuss optimizers.

Goal: Find parameters θ that minimize the loss $J(\theta)$.

Intuition: Follow the negative gradient (steepest descent direction).



The gradient $\nabla_{\theta} J(\theta)$ tells us which direction increases the loss most.
We go the **opposite direction** to decrease it.

Basic update equation:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla_{\theta} J(\theta)$$

- ▶ α is the **learning rate** (step size)
- ▶ $\nabla_{\theta} J(\theta)$ is the gradient of loss w.r.t. parameters
- ▶ Repeat until convergence (or for fixed number of steps)

Basic update equation:

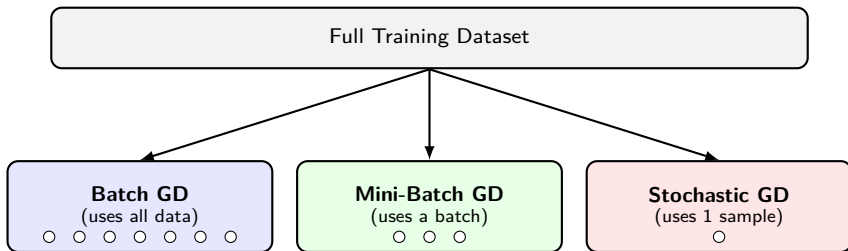
$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla_{\theta} J(\theta)$$

- ▶ α is the **learning rate** (step size)
- ▶ $\nabla_{\theta} J(\theta)$ is the gradient of loss w.r.t. parameters
- ▶ Repeat until convergence (or for fixed number of steps)

Three variants based on how much data we use:

1. **Batch Gradient Descent:** Use entire dataset
2. **Stochastic Gradient Descent (SGD):** Use one sample
3. **Mini-Batch Gradient Descent:** Use small batch of samples

How much data do we use per update (one step)?



Batch GD

- ▶ Stable updates
- ▶ Expensive per step
- ▶ Needs lots of RAM

Mini-Batch (Default)

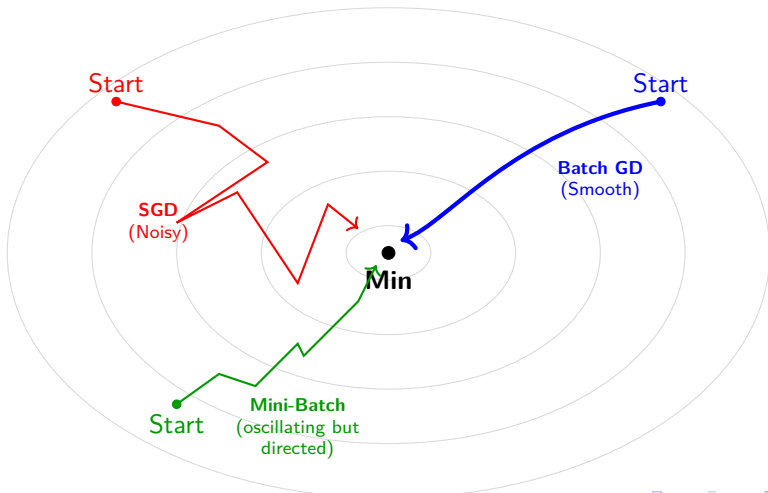
- ▶ Best of both worlds
- ▶ Stable & Fast
- ▶ GPU Efficient

Stochastic GD

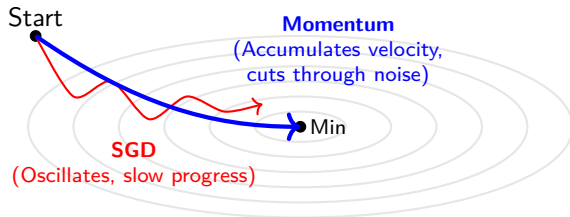
- ▶ Very noisy updates
- ▶ Can escape local minima
- ▶ Slow (can't parallelize)

In practice: we almost always use Mini-Batch (whenever GD mentioned next, it refers to the mini-batch variant).

How do they navigate the Loss Surface?



The Ravine Problem: Gradient Descent makes slow progress because it "bounces" off the steep walls.



The Solution

We add **Momentum**, which acts like a heavy ball rolling downhill, building velocity and ignoring small bumps.

It maintains a **running average** of past gradients (v_t), allowing oscillations in opposing directions to cancel out while consistent directions accumulate and accelerate.

Keep a moving average of past gradients:

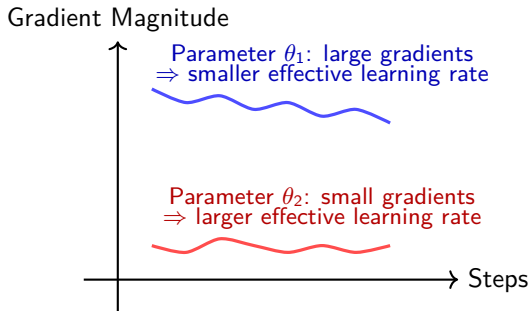
$$v_t = \underbrace{\beta v_{t-1}}_{\text{past gradients}} + \underbrace{(1 - \beta) \nabla_{\theta} J(\theta)}_{\text{new gradient}}$$

$$\theta_{t+1} = \theta_t - \alpha v_t$$

- ▶ v_t is the **velocity** (momentum term)
- ▶ β is the momentum coefficient (typically 0.9)
- ▶ α is the learning rate

Problem: Same learning rate for all parameters may not be optimal.

Idea: Give each parameter its own adaptive learning rate based on its gradient history.



Parameters with large gradients get smaller steps (learning rates).

This prevents overshooting.

Accumulate squared gradients for each parameter:

$$G_t = \underbrace{G_{t-1}}_{\text{past squared gradients}} + \underbrace{g_t^2}_{\text{new squared gradient}}$$

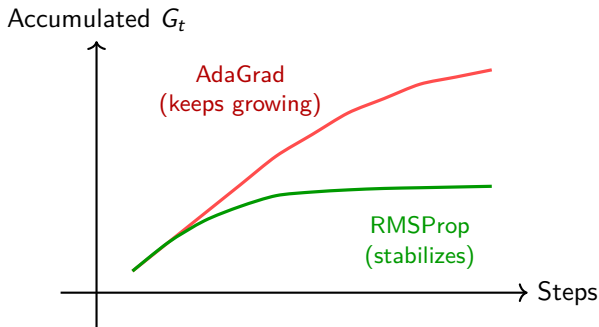
Update with parameter-wise learning rate:

$$\theta_{t+1} = \theta_t - \underbrace{\frac{\eta}{\sqrt{G_t} + \epsilon}}_{\text{adaptive learning rate}} \underbrace{g_t}_{\text{gradient}}$$

- ▶ G_t accumulates all past squared gradients $\rightarrow$ larger G_t means smaller effective learning rate
- ▶ η is the base learning rate
- ▶ ϵ is a small constant for numerical stability ($\sim 10^{-8}$)
- ▶ Each parameter gets its own effective learning rate

Problem with AdaGrad: Accumulates *all* past gradients $\rightarrow$ Learning rate can shrink too much over time (never forgets past gradients).

RMSPProp solution: Use **exponential moving average** instead (forget old gradients!).



Effect: Learning rate adapts but doesn't vanish $\rightarrow$ training continues effectively

Exponential moving average of squared gradients:

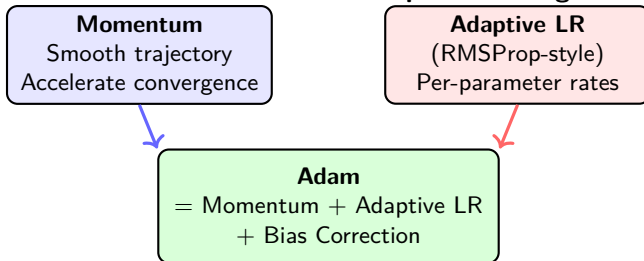
$$E[g^2]_t = \underbrace{\alpha E[g^2]_{t-1}}_{\text{decayed past gradients}} + \underbrace{(1 - \alpha)g_t^2}_{\text{new squared gradient}}$$

Update rule:

$$\theta_{t+1} = \theta_t - \underbrace{\frac{\eta}{\sqrt{E[g^2]_t} + \epsilon}}_{\text{adaptive learning rate}} \underbrace{g_t}_{\text{gradient}}$$

- ▶ α is the decay rate (typically 0.9) (old gradients gradually fade away)
- ▶ Recent gradients have more influence than old ones

Idea: Combine **momentum** + **adaptive learning rates!**



Bias Correction: At the start of training, moving averages are initialized to zero, causing them to be biased toward zero. Adam corrects this bias to get accurate estimates in early steps.

Result: Fast, stable, and works well across many problems (the most popular optimizer in deep learning!)

Maintain two moving averages:

$$m_t = \underbrace{\beta_1 m_{t-1}}_{\text{past momentum}} + \underbrace{(1 - \beta_1) g_t}_{\text{new gradient}} \quad (\text{momentum})$$

$$v_t = \underbrace{\beta_2 v_{t-1}}_{\text{past variance}} + \underbrace{(1 - \beta_2) g_t^2}_{\text{new squared gradient}} \quad (\text{adaptive LR})$$

Bias correction (fixes initial zero bias):

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

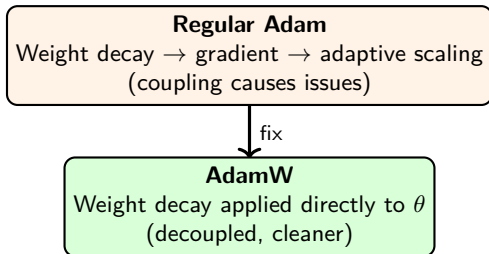
As $t \rightarrow \infty$, $\beta^t \rightarrow 0$, so correction factor $\rightarrow 1$ (no correction needed later)

Update:

$$\theta_{t+1} = \theta_t - \underbrace{\frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}}_{\text{adaptive step size}} \underbrace{\hat{m}_t}_{\text{momentum direction}}$$

Problem with Adam: Weight decay (L2 regularization) interacts poorly with adaptive learning rates.

Why? In Adam, weight decay is added to the gradient before the adaptive scaling. This means parameters with small gradients get disproportionately large weight decay, breaking the intended regularization.



Result: Better generalization, especially for large models (current best practice for training modern neural networks!)

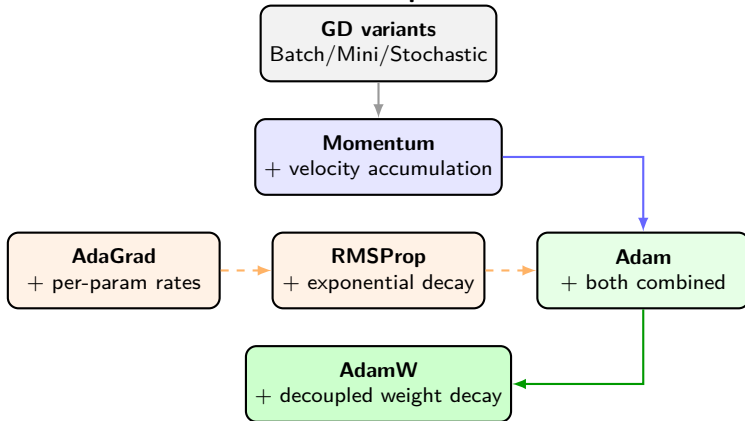
Same as Adam, plus decoupled weight decay:

$$\theta_{t+1} = \theta_t - \underbrace{\frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t}_{\text{Adam update}} - \underbrace{\eta \lambda \theta_t}_{\text{weight decay (decoupled)}}$$

- ▶ First term: standard Adam update (gradient-based optimization)
- ▶ Second term: direct weight decay applied to parameters, independent of gradients
- ▶ λ is the weight decay coefficient (typically 0.01)

Why better? Cleaner separation of optimization and regularization.

Evolution of Optimizers:



Rule of thumb: Start with **AdamW** (it's the most robust default choice!)

We said mini-batch GD is the default. How do we actually do it in PyTorch?

Naïve Approach

Manually slice the data into batches with Python loops and index math.

Error-prone, no shuffling, no parallelism.

PyTorch Way

Use Dataset + DataLoader.

Handles batching, shuffling, and parallel data loading automatically.

Two classes to know:

TensorDataset	wraps your tensors into a dataset
DataLoader	serves mini-batches from that dataset

Step 1: Wrap Data in a TensorDataset

A TensorDataset pairs inputs and targets together.

Creating a TensorDataset

```
import torch
from torch.utils.data import TensorDataset

X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32)

train_dataset = TensorDataset(X_train_tensor,
                              y_train_tensor)

# Each element is a (features, label) pair
print(train_dataset[0])  # (tensor(...), tensor(0.))
print(len(train_dataset))  # number of samples
```

Think of it like `zip(X, y)` – but for tensors.

Step 2: Create a DataLoader

A DataLoader serves mini-batches from a Dataset.

Creating a DataLoader

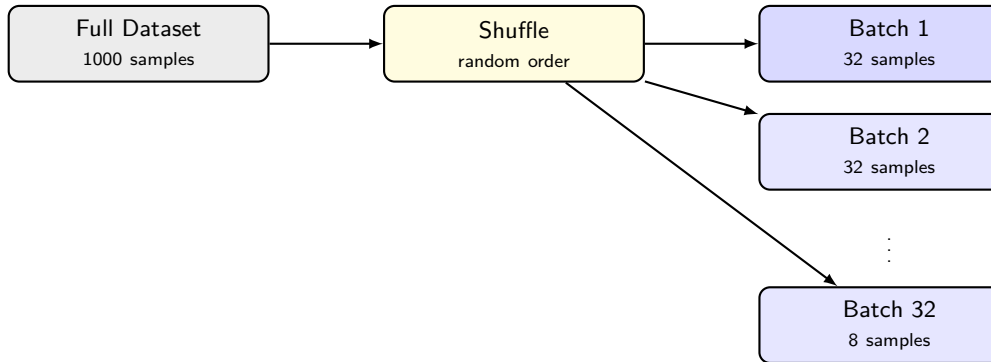
```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_dataset,
    batch_size=32,      # samples per mini-batch
    shuffle=True        # reshuffle every epoch
)

# Iterate to get one batch
X_batch, y_batch = next(iter(train_loader))
print(X_batch.shape)  # torch.Size([32, num_features])
print(y_batch.shape)  # torch.Size([32])
```

`shuffle=True` is crucial – it randomizes the order each epoch so the model doesn't memorize the sequence.

What the DataLoader Does For You



- ▶ **Shuffles** data at the start of every epoch
- ▶ **Splits** into equal-sized batches (last batch may be smaller)
- ▶ **Loads** batches lazily – memory efficient
- ▶ **Optional:** multi-worker parallel loading (num workers)

Mini-Batch Training Loop

```
model = MyMLP(input_dim=10)
criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(20):
    epoch_loss = 0
    for X_batch, y_batch in train_loader:
        # Forward pass
        predictions = model(X_batch).squeeze()
        loss = criterion(predictions, y_batch)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    epoch_loss += loss.item()
print(f"Epoch {epoch+1}, Loss: {epoch_loss:.4f}")
```

Batch size controls the trade-off between speed and noise.

Batch Size	Updates/Epoch	Speed	Gradient Quality
1 (SGD)	N	Slow	Very noisy
32	$N/32$	Fast	Good balance
128	$N/128$	Very fast	Stable
N (Full batch)	1	Fastest step	Smooth but few updates

Common choices: 32, 64, 128, 256

Start with **32** – it works well for most tasks.

Increase if training is too slow; decrease if you want more regularization.

Parameter	What It Does
<code>batch_size</code>	Number of samples per mini-batch
<code>shuffle</code>	Randomize order each epoch (True for train, False for test)
<code>drop_last</code>	Drop the last incomplete batch (useful if batch norm needs consistent sizes)
<code>num_workers</code>	Number of subprocesses for parallel data loading

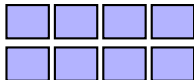
Important: Always set `shuffle=True` for training and `shuffle=False` for validation/test, so evaluation results are reproducible.

Neural network training is mostly matrix multiplications.

CPU

- ▶ Few cores (4–16), each very fast
- ▶ Great for sequential tasks
- ▶ Fine for small models and datasets

CPU



GPU

- ▶ Thousands of cores, each simpler
- ▶ Designed for parallel math
- ▶ 10–100× faster for matrix ops

GPU



GPUs have many small cores running the same operation on different data simultaneously.

PyTorch makes it easy: just move model and data to the GPU.

Step 1: Check and Set Device

```
device = torch.device('cuda' if torch.cuda.is_available()
                      else 'cpu')
print(f"Using: {device}")
```

Step 2: Move Model and Data

```
model = MyMLP(input_dim=784).to(device)

for X_batch, y_batch in train_loader:
    X_batch = X_batch.to(device)
    y_batch = y_batch.to(device)
    output = model(X_batch)
    ...
```

Key rule: Model and data must be on the **same device**. Mixing CPU tensors with a GPU model raises an error.

Scenario	CPU	GPU
Small tabular dataset (hundreds of rows)	✓	–
MLP on medium data (thousands of rows)	✓	✓
CNN on images	–	✓
Large models (millions of parameters)	–	✓
Quick debugging and prototyping	✓	–

Google Colab gives you a free GPU!

Runtime → Change runtime type → T4 GPU

Credits

KAUST Academy

King Abdullah University of Science and Technology

Original deep learning fundamentals slides contributed by Mohamed Eltayeb

KAUST–Mawhiba IOAI 2026 Training Program